

```

1  (*
2                                     CS51 Lab 12
3                               Imperative Programming and References
4  *)
5  (*
6                                     SOLUTION
7  *)
8
9  (*
10 Objective:
11
12 This lab provides practice with reference types and their use in
13 building mutable data structures and in imperative programming more
14 generally. It also gives further practice in using modules to
15 implement abstract data types.
16
17 There are three total parts to this lab. Please refer to the following
18 files to complete all exercises:
19
20     lab12_part1.ml -- Part 1: Fun with references (including `gensym`)
21     lab12_part2.ml -- Part 2: Appending mutable lists
22 -> lab12_part3.ml -- Part 3: Adding serialization to imperative queues
23 *)
24
25 (*=====
26 Part 3: Adding serialization to imperative queues
27
28 In the textbook, we defined a polymorphic imperative queue signature
29 as follows:
30
31     module type IMP_QUEUE =
32     sig
33         type 'a queue
34         val empty : unit -> 'a queue
35         val enq : 'a -> 'a queue -> unit
36         val deq : 'a queue -> 'a option
37     end
38
39 In this part, you'll add functionality to imperative queues for
40 serializing of the queue. (You added serialization to a pure
41 (immutable) stack module in Lab 8.) The signature needs to be
42 augmented first; we've done that for you here: *)
43
44 module type IMP_QUEUE =
45     sig
46         type elt
47         type queue
48
49         val empty : unit -> queue
50         val enq : elt -> queue -> unit
51         val deq : queue -> elt option
52         val to_string : queue -> string

```

```

53   end ;;
54
55   (* Notice that we've changed the module slightly so that the element
56   type (`elt`) is made explicit.
57
58   The `to_string` function needs to know how to convert the individual
59   elements to strings. That information is best communicated by
60   packaging up the element type and its own `to_string` function in a
61   module that can serve as the argument to a functor for making
62   `IMP_QUEUE`s. (You'll recall this technique from Lab 8.)
63
64   Given the ability to convert the elements to strings, the `to_string`
65   function should work by converting each element to a string in order
66   separated by arrows " -> " and with a final end marker to mark the end
67   of the queue "||". For instance, the queue generated by enqueueing, in
68   order, integer elements 1, 2, and 3 would serialize to the string
69   "1 -> 2 -> 3 -> ||" (as shown in the example below).
70
71   We've provided a functor for making imperative queues, which works
72   almost identically to the final implementation from the book (based on
73   mutable lists) except for abstracting out the element type in the
74   functor argument.
75
76   .....
77   Exercise 11: Your job is to complete the implementation by finishing
78   the `to_string` function. (Read on below for an example of the use of
79   the functor.)
80   .....*)
81
82   module MakeImpQueue (Elt : sig
83       type t
84       val to_string : t -> string
85     end)
86       : (IMP_QUEUE with type elt = Elt.t) =
87   struct
88     type elt = Elt.t
89     type mlist = mlist_internal ref
90     and mlist_internal =
91       | Nil | Cons of elt * mlist
92     type queue = {front: mlist; rear: mlist}
93
94     let empty () = {front = ref Nil; rear = ref Nil}
95
96     let enq x q =
97       match !(q.rear) with
98       | Cons (_hd, tl) -> assert (!tl = Nil);
99                           tl := Cons(x, ref Nil);
100                           q.rear := !tl
101       | Nil -> assert (!(q.front) = Nil);
102                q.front := Cons(x, ref Nil);
103                q.rear := !(q.front)
104
105     let deq q =
106       match !(q.front) with

```

```

107         | Cons (hd, tl) ->
108             q.front := !tl ;
109             (match !tl with
110             | Nil -> q.rear := Nil
111             | Cons(_, _) -> ());
112             Some hd
113         | Nil -> None
114
115     (* our solution for defining `to_string`: *)
116     let to_string {front; _} =
117         let rec to_string' mlst =
118             match !mlst with
119             | Nil -> "||"
120             | Cons (hd, tl) ->
121                 Printf.sprintf "%s -> %s" (Elt.to_string hd) (to_string' tl) in
122         to_string' front
123     (* end of our solution *)
124
125 end ;;
126
127 (* To build an imperative queue, we apply the functor to an
128 appropriate argument. For instance, we can make an integer queue
129 module: *)
130
131 module IntQueue = MakeImpQueue (struct
132     type t = int
133     let to_string = string_of_int
134 end) ;;
135
136 (* And now we can test it by enqueueing some elements and converting
137 the queue to a string to make sure that the right elements are in
138 there. *)
139
140 let test () =
141     let open IntQueue in
142     let q = empty () in
143     enq 1 q;
144     enq 2 q;
145     enq 3 q;
146     to_string q ;;
147
148 (* Running the `test` function should have the following behavior:
149
150     # test () ;;
151     - : string = "1 -> 2 -> 3 -> ||"
152
153 *)

```